

API Security Risk Analysis

Future Interns Cybersecurity Internship – Task 3

Project: API Security Risk Analysis (Modern SaaS Skill)

Assessment Target: JSONPlaceholder Public Demo API

Base URL: <https://jsonplaceholder.typicode.com>

Assessment Type: Safe, read-only API security analysis

1. Executive Summary

This report documents an API security assessment conducted as part of the Future Interns Cybersecurity Internship Task 3.

The assessment focused on identifying common API security risks and understanding how authentication, authorization, data exposure, HTTP security headers, rate limiting, and endpoint design affect API security.

The assessment used JSONPlaceholder, a publicly available demonstration REST API designed for development and testing.

Six read-only GET endpoints were tested:

- `/users`
- `/users/1`
- `/posts`
- `/posts/1`
- `/posts?userId=1`
- `/posts/1/comments`

The assessment identified several contextual security observations, including public unauthenticated endpoint access, broad demonstration-data responses, predictable resource identifiers, and technology disclosure through the `X-Powered-By` response header.

The assessment also identified positive security controls including HTTPS, `X-Content-Type-Options: nosniff`, JSON content-type declaration, and rate-limit response headers.

No High-severity or Medium-severity vulnerability was established.

No authentication bypass, authorization bypass, IDOR exploitation, injection attack, denial-of-service activity, or destructive operation was performed.

2. Objective

The objectives of this assessment were to:

1. Analyze a public/test API.
2. Identify common API security risks.
3. Review authentication requirements.
4. Review authorization considerations.
5. Inspect API response data.
6. Examine HTTP security headers.
7. Observe rate-limiting indicators.
8. Identify possible excessive data exposure.

9. Examine safe query-parameter behavior.
 10. Classify observations according to severity.
 11. Explain potential business impact.
 12. Recommend appropriate remediation measures.
-

3. Scope

The assessment was limited to the public JSONPlaceholder API.

In Scope

- Public API endpoints
- HTTP GET requests
- Response status codes
- Response headers
- Response data
- Authentication requirements
- Authorization considerations
- Query parameters
- Rate-limit indicators
- Security headers
- Data exposure observations

Out of Scope

- Private systems
 - Production systems
 - Credential attacks
 - Authentication bypass
 - Authorization bypass
 - IDOR exploitation
 - Brute force
 - Denial-of-service testing
 - API flooding
 - Destructive requests
 - Malicious injection payloads
 - Real customer data
-

4. Ethical Testing Methodology

The assessment followed a safe and non-destructive methodology.

Only read-only GET requests were used.

No attempt was made to exploit vulnerabilities.

The following activities were intentionally excluded:

- Credential theft
- Password attacks
- Authentication bypass
- Authorization bypass

- SQL injection
- Cross-site scripting
- Command injection
- Path traversal
- Fuzzing
- Denial-of-service
- Data modification
- Data deletion

The goal was to understand observable API security behavior without causing harm.

5. API Tested

JSONPlaceholder

JSONPlaceholder is a public fake REST API used for development and testing.

Base URL:

`https://jsonplaceholder.typicode.com`

The API provides demonstration resources such as users, posts, and comments.

Because the API is intentionally public and contains fake data, findings relating to authentication and data exposure must be interpreted as contextual observations rather than confirmed vulnerabilities in a production environment.

6. Tools Used

The assessment used command-line HTTP requests and response inspection.

Primary techniques included:

- `curl`
- HTTP response inspection
- JSON response inspection
- Header inspection
- Local evidence capture
- Markdown documentation

The collected evidence was stored in:

`FUTURE_CS_Task3/evidence/`

7. Test Cases

Test	Method	Endpoint	Result
01	GET	<code>/users</code>	200 OK
02	GET	<code>/users/1</code>	200 OK
03	GET	<code>/posts</code>	200 OK
04	GET	<code>/posts/1</code>	200 OK

Test	Method	Endpoint	Result
05	GET	/posts?userId=1	200 OK
06	GET	/posts/1/comments	200 OK

All requests were read-only.

8. Finding 1 — Public Unauthenticated Endpoint Exposure

Description

The tested endpoints were accessible without an API key or authentication token.

For example:

GET /users

returned a successful response without an Authorization header.

Observation

This is expected behavior for JSONPlaceholder because it is specifically designed as a public demonstration API.

Therefore, this observation is not treated as a confirmed vulnerability.

However, a production SaaS API containing private information should not expose protected resources without appropriate authentication.

Severity

Low — Contextual

Potential Business Impact

If replicated in a production environment containing private information, unrestricted endpoint access could result in unauthorized information exposure.

Potential impacts could include:

- Data exposure
- Privacy violations
- Unauthorized information gathering
- Increased attack surface

Recommendation

For production APIs:

- Require authentication for protected endpoints.
- Validate authentication tokens server-side.
- Use secure token handling.
- Separate public and private API resources.
- Apply authorization after authentication.

9. Finding 2 — Broad User Data Exposure

Endpoint

GET /users

Description

The endpoint returned multiple fields for each demonstration user.

Fields included:

- id
- name
- username
- email
- address
- phone
- website
- company

Observation

The information is fake demonstration data.

However, returning unnecessary fields from production APIs can result in excessive data exposure.

Severity

Low — Contextual

Potential Business Impact

If sensitive production information were unnecessarily returned, an attacker could obtain information beyond what is required for the intended application function.

Potential impacts include:

- Increased privacy exposure
- Information disclosure
- Unnecessary attack surface
- Data protection concerns

Recommendation

Production APIs should:

- Return only required fields.
 - Apply field-level authorization.
 - Use strict response schemas.
 - Avoid returning internal database fields.
 - Apply data minimization principles.
-

10. Finding 3 — Technology Disclosure

Observed Header

X-Powered-By: Express

Description

The response indicates that the server uses Express.

Technology disclosure is not automatically a vulnerability.

However, unnecessary framework information can assist reconnaissance.

Severity

Informational

Potential Business Impact

An attacker may use technology information to identify potential framework-specific attack surfaces.

The actual security impact depends on the rest of the application configuration.

Recommendation

Where practical, production applications should remove unnecessary technology-identification headers.

11. Finding 4 — Predictable Resource Identifiers

Examples:

/users/1

/posts/1

Description

The API uses numeric resource identifiers.

Predictable identifiers can make resources easy to enumerate.

However, predictable identifiers alone do not constitute an authorization vulnerability.

Severity

Informational / Contextual

Testing Limitation

No authorization bypass or IDOR exploitation was attempted.

There was no authenticated multi-user environment available for comparison.

Recommendation

Production applications should never rely on unpredictable identifiers as the primary security mechanism.

Instead:

- Authenticate users.
 - Authorize every object request.
 - Perform server-side ownership checks.
 - Use object-level authorization controls.
-

12. Finding 5 — Query Parameter Behavior

Endpoint

GET /posts?userId=1

Description

A safe documented query parameter was supplied to observe API filtering behavior.

The API returned posts associated with `userId: 1`.

Severity

Informational

Security Interpretation

This demonstrates normal parameterized API behavior.

No malicious payloads were used.

No attempt was made to manipulate the parameter to access unauthorized data.

Recommendation

Production APIs should:

- Validate all input server-side.
 - Apply strict schemas.
 - Authorize user-specific queries.
 - Reject invalid input.
 - Avoid trusting client-provided identifiers.
-

13. Positive Security Control — X-Content-Type-Options

Observed:

X-Content-Type-Options: nosniff

Significance

This header helps prevent browsers from MIME-sniffing responses away from their declared content type.

Assessment

Positive Security Control

The header should be maintained in production deployments.

14. Positive Security Control — Rate-Limit Headers

Observed headers included:

X-RateLimit-Limit

X-RateLimit-Remaining

X-RateLimit-Reset

Significance

The presence of rate-limit information indicates that rate-limiting controls are represented by the API infrastructure.

Rate limiting helps protect APIs against:

- Excessive automated requests
- Resource exhaustion
- Abuse
- Brute-force attempts
- Excessive scraping

Assessment

Positive Control Observed

No high-volume testing was performed.

15. Positive Security Control — HTTPS

The API was accessed using HTTPS.

Security Significance

HTTPS protects API traffic during transmission by providing encrypted communication between client and server.

Assessment

Positive Security Control

Production APIs should enforce HTTPS for all sensitive communication.

16. Cache Behavior

The API returned cache-related headers including:

`Cache-Control: max-age=43200`

Security Consideration

Caching can improve performance.

However, production APIs containing private or user-specific data must carefully control caching behavior.

Sensitive responses should not be accidentally served to unauthorized users through intermediary or browser caches.

Assessment

Informational Observation

The JSONPlaceholder records are demonstration data.

17. Authentication Assessment

The tested API did not require an API key or authentication token.

Assessment

For JSONPlaceholder, this is expected.

The API is intended for public development and testing.

Therefore, the absence of authentication is not classified as a confirmed vulnerability.

Production Recommendation

A real SaaS API should:

- Authenticate protected users.
 - Validate tokens securely.
 - Use appropriate token expiration.
 - Protect authentication credentials.
 - Separate public resources from protected resources.
-

18. Authorization Assessment

Comprehensive authorization testing could not be performed because there were no private authenticated accounts available.

The following were not attempted:

- Authorization bypass
- IDOR exploitation
- Privilege escalation
- Session manipulation
- Accessing another user's private information

Assessment

Not Fully Testable Within Demonstration Scope

Production Recommendation

Every protected API request should verify that the authenticated user is authorized to access the requested object.

19. Input Validation Assessment

The safe query parameter:

`userId=1`

was tested.

The API returned the expected filtered demonstration records.

No malicious payloads were submitted.

Therefore, a complete input-validation assessment was outside the scope of this task.

Assessment

Limited Assessment

Production applications should validate and sanitize all API input server-side.

20. Risk Summary

Finding	Severity	Business Impact	Recommended Remediation
Public unauthenticated demo endpoints	Low / Contextual	Could expose protected information if replicated in production	Require authentication for protected resources
Broad user-object response	Low / Contextual	Could expose unnecessary information	Apply data minimization and field filtering
X-Powered-By disclosure	Informational	Provides framework information	Remove unnecessary technology disclosure
Predictable numeric IDs	Informational	Could allow enumeration if authorization is weak	Enforce object-level authorization
Query parameter behavior	Informational	Unsafe handling could expose data in production	Validate input and enforce authorization
Rate-limit headers	Positive Control	Helps indicate rate limiting	Maintain and monitor limits
X-Content-Type-Options	Positive Control	Reduces MIME-sniffing risk	Maintain header

Finding	Severity	Business Impact	Recommended Remediation
HTTPS	Positive Control	Protects data in transit	Continue enforcing HTTPS

21. Severity Definitions

High

A confirmed issue with potential for serious unauthorized access, major data compromise, account compromise, or significant business impact.

No High-severity vulnerability was established.

Medium

A security weakness that could produce meaningful impact under realistic conditions.

No Medium-severity vulnerability was established.

Low

A limited or contextual security concern that could become more significant in a production environment.

Informational

An observation useful for security awareness that does not represent a confirmed vulnerability.

22. Evidence Collected

The following evidence files were created:

- `users_response.txt`
- `user_1_response.txt`
- `posts_response.txt`
- `posts_user1_response.txt`
- `post_1_response.txt`
- `post_1_comments_response.txt`

They are located in:

`FUTURE_CS_Task3/evidence/`

Additional documentation includes:

- `analysis/endpoint_analysis.md`
- `analysis/risk_analysis.md`
- `tests/requests.md`
- `references/sources.md`
- `evidence/README.md`

23. Evidence Integrity

The evidence files contain the API responses captured during the assessment.

They document the observable behavior of the tested endpoints at the time of testing.

The evidence should be interpreted together with the assessment methodology and limitations.

The collected responses should not be interpreted as proof that JSONPlaceholder represents the security posture of a real production SaaS platform.

24. Assessment Limitations

The following limitations apply:

1. JSONPlaceholder is a public demonstration API.
 2. The API uses fake demonstration data.
 3. No real customer information was accessed.
 4. No authenticated multi-user environment was available.
 5. Authorization testing was therefore limited.
 6. No exploitation was performed.
 7. No high-volume rate-limit testing was performed.
 8. No malicious input payloads were submitted.
 9. No destructive operations were performed.
 10. The assessment was a focused internship exercise rather than a complete penetration test.
-

25. Production Security Recommendations

For a real SaaS API, the following security controls are recommended:

1. Implement strong authentication.
 2. Enforce server-side authorization.
 3. Apply object-level authorization.
 4. Minimize API response data.
 5. Apply field-level access control.
 6. Implement rate limiting.
 7. Monitor API activity.
 8. Validate all input.
 9. Use strict API schemas.
 10. Remove unnecessary technology-disclosure headers.
 11. Review cache behavior.
 12. Enforce HTTPS.
 13. Maintain security logging.
 14. Monitor suspicious activity.
 15. Regularly review API endpoints.
 16. Follow OWASP API Security guidance.
 17. Conduct periodic security assessments.
-

26. Overall Assessment Conclusion

The API security assessment successfully demonstrated a controlled methodology for reviewing a public REST API.

Six endpoints were tested using safe read-only requests.

The assessment identified contextual observations involving:

- Public endpoint exposure
- Broad demonstration-data responses
- Predictable resource identifiers
- Technology disclosure
- Query parameter behavior
- Cache behavior

The assessment also identified positive controls involving:

- HTTPS
- JSON content type
- `X-Content-Type-Options: nosniff`
- Rate-limit headers

No High-severity or Medium-severity vulnerability was established.

No authentication bypass, authorization bypass, IDOR exploitation, injection attack, denial-of-service activity, or destructive operation was performed.

The findings should therefore be interpreted as an educational API security assessment and production-design review rather than as a complete penetration test.

27. References

The assessment methodology was informed by:

- OWASP API Security Project
- API Security Checklist
- Public APIs repository
- JSONPlaceholder documentation
- Future Interns Task 3 requirements
- HTTP security header guidance
- Rate-limiting security practices

Detailed references are provided in:

`FUTURE_CS_Task3/references/sources.md`

End of Report